

Exato, agora fechou a lógica. Deixa eu confirmar que entendi o teu raciocínio antes de codar, porque tem uma sutileza importante no que você disse:

(A) sozinho não pode porque depende da densidade de questões. Se você corta tudo com `b < limiar`, uma habilidade que tem poucas questões difíceis no banco deixa o aluno sem material suficiente pra provar domínio — o corte esvazia a amostra. Não dá pra exigir "acerte 8 questões $b > 0.7$ " se só existem 3 no grafo.

(B) é o padrão — peso contínuo, todo acerto conta proporcional à dificuldade. Funciona sempre, independente de quantas questões existem, porque não descarta amostra, só pondera.

(C) é o refinamento condicional — *quando há muitas questões*, aí sim você pode se dar ao luxo de aplicar o corte de elegibilidade por cima do peso, porque a amostra aguenta. Foi o que você falou antes: "se houver muitas questões é o que eu falei".

Então a função é adaptativa à densidade:

```
se densidade_de_questões(habilidade, b > limiar_meta) >= mínimo_amostrai:
    modo = C    # corte de elegibilidade + peso contínuo dentro do corte
senão:
    modo = B    # só peso contínuo, usa todas as questões
```

Isso é elegante e é a decisão certa de engenharia — o modo degrada de C pra B quando a amostra não aguenta o corte, em vez de quebrar. **Isso** é adaptação real, não o "degradar graciosamente" que você (com razão) odiou antes; aqui é a matemática se ajustando à realidade dos dados, não esconder erro.

Duas coisas que eu ainda preciso pra escrever a fórmula fechada e o Cypher certo — e aí produzo direto:

Primeiro: o **peso contínuo (B)** — quando você diz "acerto vale proporcional à dificuldade", é proporcional ao **param-b puro** (acertar $b=0.9$ vale 0.9), ou passa pela **curva do TRI** (a probabilidade de acerto dado a proficiência-alvo da meta, tipo a 3PL/2PL)? Porque teu mestrado é TRI — suspeito que você queira a curva, não o b cru. A diferença: b cru é linear; a curva logística pondera pela *surpresa* (acertar algo que a tua proficiência-alvo diz que você erraria vale muito mais).

Segundo: o **mínimo amostral** e o **limiar meta** — são parâmetros que você já tem calibrados (do mestrado / dos microdados), ou são knobs que a gente expõe (tipo os do painel de Configuração Temporal) pro aluno/sistema ajustar?

Responde esses dois e eu escrevo o **PhaseTransitionService** completo + o Cypher, sem chutar a parte que é teu trabalho acadêmico. Manda.

me alinha primeiro aqui o que vc ta pensando emcodar

Boa, vamos alinhar antes de eu escrever uma linha. Aqui está o que estou pensando produzir, em ordem — me corrige onde eu estiver assumindo errado.

Peça 1 — **PhaseTransitionService (Dart, no serverpod).** Uma função pura, sem estado, sem LLM. Entra: desempenho do aluno numa habilidade, a meta dele, os param-b (ou a curva TRI) das questões que ele respondeu, e o peso do mapa de calor daquela habilidade. Sai: uma das 4 fases (**INICIA/PROGRIDE/AVANCA/DOMINA**). Determinística — mesmo input, mesma fase, sempre. Dentro dela mora a lógica adaptativa que você acabou de definir: checka a densidade de questões elegíveis; se aguenta o corte, roda modo C; se não, modo B.

Peça 2 — o Cypher que alimenta a Peça 1. Uma query que, dado (**aluno, habilidade**), puxa do grafo: as questões que o aluno respondeu naquela habilidade com seus param-b e se acertou, a meta do aluno, e o peso do mapa de calor. É o que transforma o grafo na "medição" do controlador. Só leitura.

Peça 3 — o ponto de escrita de volta. Onde a fase calculada volta pro grafo (a aresta de fase do aluno). Isso é o "Mestre escreve o resultado" fechando a malha. Mas aqui eu ainda **não** vou codar até a gente ter as Peças 1 e 2 sólidas — é o passo do loop, vem depois.

O que eu **não** vou fazer agora, de propósito, pra não misturar frentes: não vou tocar na antecipação, nem no rating de surfaces, nem no Agente Mestre. A fase é o tijolo que sustenta todos eles; construo ela isolada, testável sozinha, e só depois ligo nos outros.

Três coisas que eu quero cravar contigo antes de codar, porque são onde eu poderia inventar:

Primeiro — **onde essa função roda?** Minha aposta: no serverpod (Dart), porque é lá que a antecipação e o resto vão consumir. O grafo só *fornece os dados* (via Cypher), o *cálculo* é no Dart. Alternativa seria fazer o cálculo dentro do próprio Cypher (Neo4j calculando). Eu acho Dart melhor — mais testável, e você não quer lógica de negócio presa no banco. Concorde, ou você prefere no grafo?

Segundo — **a fórmula do peso (B)**. Ainda preciso saber se é param-b puro ou a curva TRI (a pergunta que fiz). Isso é o miolo matemático; sem isso a Peça 1 fica com um buraco no meio.

Terceiro — **a forma dos dados no grafo**. Preciso dos nomes reais: como a meta do aluno está guardada, como a relação de resposta (**RESPONDEU** ?) marca acerto e liga na questão, onde está o param-b na **Questao**, e onde está o peso do mapa de calor. Sem os nomes certos, o Cypher é chute.

Antes de você responder tudo: faz sentido essa ordem — função pura no Dart primeiro, Cypher alimentando, escrita-de-volta depois? E o cálculo é no serverpod mesmo, não no grafo? Alinha isso comigo e depois a gente resolve a matemática e os nomes.

Peça 1 -> nao , vc nem tem os dados direito, vai enche de placehodler e me datralho.

Peça 2-> nao, se nao tem isso no grafo, vc vai cria sem acesso ao servidor pra ver os paamtros?

_> cada token vale ouro vou pedi primeiro um arquivo .md com uma descricao completa e pontuas de todos os modulos do sistema

um diagrama de controle com a descricao que vc fez do sistema deste ponto de vista em arquivo .puml

um arquivo NEXT_STEPS.md no qual vc vai, a partir examtante do codigo novo do serverpod que vc fez, definir o sproximos passos para fazer a interligacao do serverpod e registrar no grafo sempre que uma nova superficie for registrada no psql (só novas,qdo ele usar existentes nao precisa)

é isso. tres aquivos.

Perfeito, muito mais sensato — documentar o que existe de verdade antes de codar em cima de dados que eu não vi. Três arquivos, sem placeholder, ancorado no código real. Deixa eu reler rápido o endpoint que a gente fez pra o NEXT_STEPS sair preciso, e produzo os três.

Verify module files still present and key methods for NEXT_STEPS accuracy

